Project Millipede: WASM Redaction & Execution Roadmap

This document covers our client-side WebAssembly pseudonymization engine, the SolidJS frontend, and our 4-stage execution roadmap.

🧩 Browser-Side Rust WebAssembly Redaction

To sanitize 1:1 notes instantly without causing UI lag, the SolidJS frontend offloads raw text processing to a background thread running a compiled WebAssembly binary:

```
// Example worker wrapper in SolidJS
import WASM_MODULE from './wasm/redact_core_bg.wasm';
import init, { redact_pii_deterministic } from './wasm/redact_core';

self.onmessage = async (event) => {
    const { rawText, seed } = event.data;

    // Initialize WebAssembly linear memory
    await init(WASM_MODULE);

    // Process text at native compilation speed
    const sanitizedResult = redact_pii_deterministic(rawText, seed);

    // Send sanitized text back to the main thread
    self.postMessage(sanitizedResult);
};
```

🎨 SolidJS Direct DOM Dashboard Strategy

We choose **SolidJS** for its direct DOM compilation, making it ideal for high-performance WebAssembly and Web Component integration.

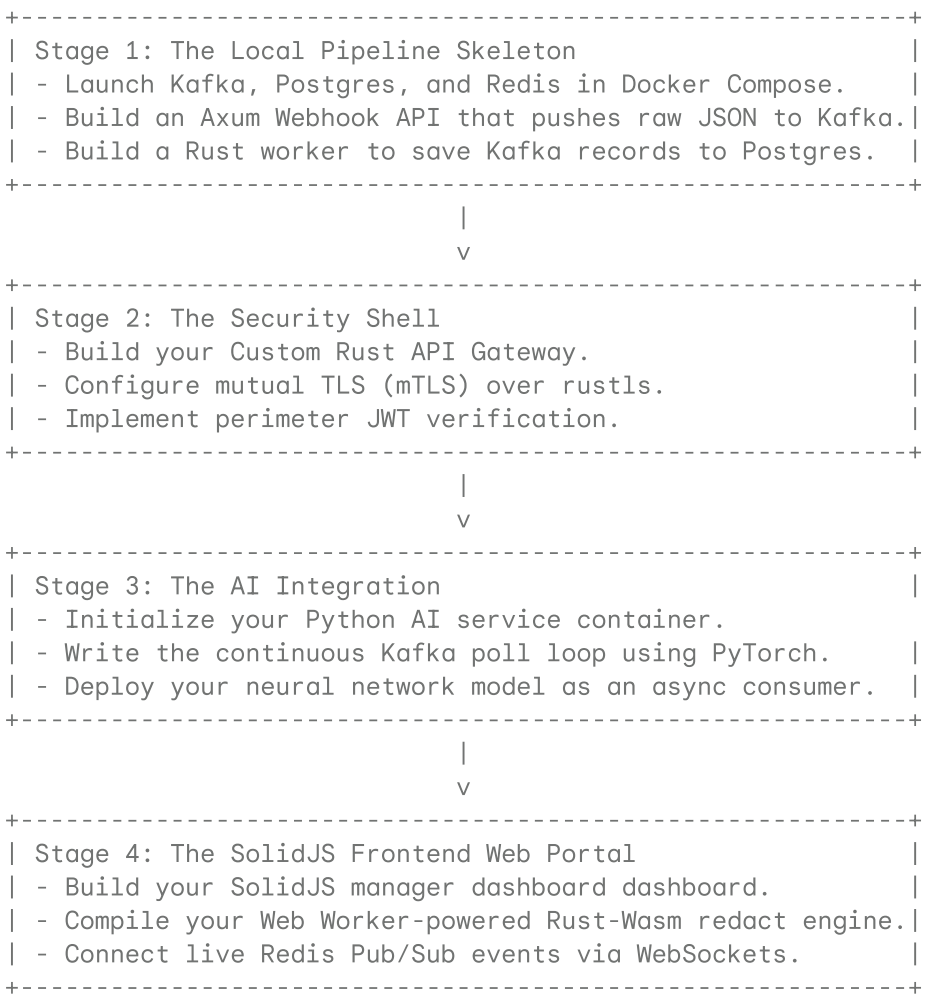
```
        [ Signal Update ]
                |
                ▼ (Direct DOM Node Update)
+-------------------------------+
|   Browser Document Object     |
|   - Real DOM modified directly|
|   - Zero VDOM reconciliation  |
+-------------------------------+
```

Because SolidJS compiles down to real, raw DOM nodes:

- We pass data references smoothly to Rust-Wasm memories.
- Traditional charting frameworks (like **ApexCharts** or **Chart.js**) can bind directly to raw DOM references in SolidJS's native `onMount` hooks.

🚀 The 4-Stage Execution Roadmap

To keep development manageable and prevent scope creep, we will build `millipede` in 4 clear, incremental phases:


Stage 1: The Local Pipeline Skeleton
- Launch Kafka, Postgres, and Redis in Docker Compose.
- Build an Axum Webhook API that pushes raw JSON to Kafka.
- Build a Rust worker to save Kafka records to Postgres.
Stage 2: The Security Shell
- Build your Custom Rust API Gateway.
- Configure mutual TLS (mTLS) over rustls.
- Implement perimeter JWT verification.
Stage 3: The AI Integration
- Initialize your Python AI service container.
- Write the continuous Kafka poll loop using PyTorch.
- Deploy your neural network model as an async consumer.
Stage 4: The SolidJS Frontend Web Portal
- Build your SolidJS manager dashboard dashboard.
- Compile your Web Worker-powered Rust-Wasm redact engine.
- Connect live Redis Pub/Sub events via WebSockets.